Tp1_Jonathan_David_Manga

MDJ_007

2026-04-13

1 Exercie 1

question1

Variable	Type	Mode
x	Vecteur numerique	Integer[1]
y	vecteur logique	logical
z	Vecteur Texte	character
w	liste	list

question 2

Pour le premier cas x+y

R va transformer le vecteur y en numerique avec **TRUE =1 et FALSE = 0**

le resultat final est le vecteur :

```
resultat <- c(2,2,4)
```

Pour le second cas x + z Le resultat est une erreur

Pour le troisieme cas R transforme le vecteur z en numeric des lors nous avons une addition

le resultat est :

```
resultat <- c(5,7,9)
```

Pour le quatrieme cas :

La fonction unlist(w) aplatit la liste en un vecteur. À cause de la présence de "2" (texte) dans la liste, R transforme tout le contenu de w en texte lors de l'aplatissement. On retombe sur le cas Vecteur texte + Vecteur réel, ce qui provoque une erreur.

[1] stocke neanmoins comme Double

question 3 et 4

Expression	Coercition implicite ?	Type cible	Logique de conversion mobilisée
$x + y$	**OUI**	`double` (Réel)	R transforme les logiques en nombres (`TRUE`=1, `FALSE`=0) pour permettre l'opération arithmétique.
$x + z$	**NON** (Erreur)	Aucun	R refuse de transformer implicitement du texte en nombre pour une addition.
$as.numeric(z) + x$	**NON** (Explicite)	`double` (Réel)	La conversion est forcée par l'utilisateur. R additionne ensuite deux vecteurs de même type.
$unlist(w) + x$	**OUI**	`character` (Texte)	Pour créer un vecteur à partir d'une liste hétérogène, R contraint tous les éléments vers le type le plus flexible (le texte).

question 5

Executer ce code menera a une erreur. Les valeurs tels que "10" et "20" seront remplaces par leur equivalent numerique. Cependant Tenter de transformer le "A" menera a une erreur :Le "A" sera remplace par un NA

question 6

```r
# On propose le vecteur et l'operation a effectue et on utilise
#la fonction wich combine avec is.Na()


#Par exemple


# Le vecteur à tester
v <- c("10", "A", "20")

# 1. On tente la conversion
v_num <- as.numeric(v)

# 2. On détecte les indices qui sont devenus NA à cause de la conversion
# (en excluant les NA qui étaient déjà peut-être là)
problemes <- which(is.na(v_num) & !is.na(v))

# 3. Test de robustesse
if(length(problemes) > 0) {
  print("Attention : Des données non numériques ont été détectées !")
  print(paste("Éléments fautifs :", v[problemes]))
} else {
  print("Conversion réussie : toutes les données sont numériques.")
}
```

2 Exercice 2

question1

\begin{center}

Colonne	Type
A	Vecteur numerique
B	vecteur texte
C	Vecteur logique

question 2

Il s'agit du dataframe ici chaque colonne peut avoir un type different . C'est semblable a un tableau sur une population de 3 individus avec 3 caracteres

question 3 et 4

Ce qu'il va se passer c'est que R va tout transformer dans le type dominant qui ici s'agit de character . Pour une matrice en effet toutes les colonnes doivent etre de meme type c'est la principale difference de comportement

comme le montre le code ci dessous

```
A <- c(1, 2, 3)
B <- c("a", "b", "c")
C <- c(TRUE, FALSE, TRUE)
D <- data.frame(A, B, C)
as.matrix(D)
```

```
##      A   B   C
## [1,] "1" "a" "TRUE"
## [2,] "2" "b" "FALSE"
## [3,] "3" "c" "TRUE"
```

```
typeof(as.matrix(D))
```

```
## [1] "character"
```

Question 5

Lorsque l'on va tranformer on pourra plus faire les operations naturels les transformation de variable vu que tout est en texte . Certains graphiques ne pourront plus etre effectue notamment ceux sur le variables continues

question 6

Sans pour autant modifier le type on peut appliquer un filtre sur les colonnes

```r
# On définit le filtre : "Est-ce que la colonne est numérique ?"
mon_filtre <- sapply(df, is.numeric)
#colonne logique
mon_filtre2 <- sapply(df,is.logical)
#colonne de texte
mon_filtre3 <- sapply(df,is.character)
# On applique le filtre sur les colonnes (après la virgule)
df_propre <- df[, mon_filtre]
```

3 Exercice 3

Question 1

Pour respecter la hiérarchie logique, on définit les *levels* explicitement :

```r
niveau <- factor(niveau, levels = c("faible", "moyen", "élevé"), ordered = TRUE)
```

Question 2

R stocke les facteurs sous forme d'entiers. Ici : **faible = 1, moyen = 2, élevé = 3**. Le vecteur interne est donc $(1, 2, 3, 2, 1)$.

Question 3

L'instruction `as.numeric(niveau)` est dangereuse car elle retourne les codes d'indexation internes (1, 2, 3) et non les valeurs réelles. Dans une analyse, cela peut conduire à traiter des rangs arbitraires comme des grandeurs mathématiques, faussant ainsi les calculs de moyenne ou de corrélation.

question 5

Cette operation n'a pas de sens on effectue en general la moyenne sur plusieurs valeurs ici l'operation va juste extraire 300 et calculer la moyrnnr dessus ce qui nous redonne le meme nombre .

question6

Si ils sont mal ordonnes on aura des difficultes dans l'analyse notemment dans un graphique en barre qui va representer une information fausse.

4 Exercice 4

```r
# Création de la cohorte d'étudiants
cohorte <- data.frame(
  id = c(101, 102, 103, 104, 105),

  # Conversion appropriée des variables qualitatives (Facteurs)
  sexe = factor(c("M", "F", "F", "M", "F")),
```

```r
  filiere = factor(c("ISEP", "ISE", "AS", "ISEP", "ISE")),

  # Typage explicite des variables numériques
  s1 = c(14.5, 12.0, 15.2, 10.8, 15),
  s2 = c(13.0, 11.5, 18, 12.4, 14.0),

  boursier = c(TRUE, FALSE, TRUE, TRUE, FALSE), # Logique

  # Gestion des valeurs manquantes pour la région
  region = c("Dakar", "Thiès", NA, "Saint-Louis", "Dakar")
)
cohorte$id <- factor(cohorte$id)
```

question2

```r
cohorte$moyenne_annuelle <- rowMeans(cohorte[, c("s1", "s2")])
```

question3

```r
cohorte$decision <- ifelse(cohorte$moyenne_annuelle >= 12, "Admis", "Ajourné")


cohorte$decision <- factor(cohorte$decision)
```

question4

On pourrait penser a transformer la variable filiere en factor car il existe un lien logique entre ISEP et ISE . Cependant tel n'est pas le cas avec AS . C'est une filiere independante des deux autres. Du coup il n'est vraiment pas necessaire de le faire.

question 5

L'analyse de la structure confirme un typage rigoureux : les variables qualitatives (id, sexe, filière, décision) sont traitées en facteurs, tandis que les moyennes conservent leur format numérique (num). La cohérence est totale, car chaque identifiant est unique et les statuts de réussite valident strictement la règle des 12 points. L'unique anomalie identifiée est une donnée manquante (NA) pour la région, ce qui ne compromet pas la fiabilité des calculs statistiques.

```r
summary(cohorte)
str(cohorte)
```

question6

```r
# 1. Contrôle des types (on vérifie si des colonnes numériques ont des problèmes)
type_check <- !is.numeric(cohorte$s1) | !is.numeric(cohorte$s2)

# 2. Détection des notes aberrantes (hors 0-20)
```

```r
aberrantes <- which(cohorte$s1 < 0 | cohorte$s1 > 20 | cohorte$s2 < 0 | cohorte$s2 > 20)

# 3. Détection des NA critiques (ID ou Moyenne manquante)
na_critiques <- which(is.na(cohorte$id) | is.na(cohorte$moyenne_annuelle))

# Affichage des alertes
if(length(aberrantes) > 0) print(paste("Note aberrante à la ligne :", aberrantes))
if(length(na_critiques) > 0) print(paste("Donnée critique manquante à la ligne :", na_critiques))
```

question 7

Le choix du type double pour les moyennes est idéal pour la régression linéaire, car il permet de traiter ces variables de manière continue sans perte de précision mathématique. N'ayant encore qu'une comprehension superficielle de l'analyse factorielle je ne trouve pas encore l'importance du typage ici.

question du cours precedent

```r
# 1. La matrice à diagonaliser
A <- matrix(c(4, 1, 2, 3), 2, 2, byrow = TRUE)

# 2. Valeurs propres (Calcul du discriminant de det(A - lambda*I) = 0)
# Pour une matrice [[a, b], [c, d]], le polynôme est : lambda^2 - Tr(A)lambda + det(A)
tr_A <- sum(diag(A))
det_A <- det(A)

# Résolution de : lambda^2 - tr_A*lambda + det_A = 0
delta <- tr_A^2 - 4 * det_A
val_propres <- c((tr_A + sqrt(delta)) / 2, (tr_A - sqrt(delta)) / 2)

# 3. Vecteurs propres (Résolution de (A - lambda*I)X = 0)
# Pour chaque lambda, on cherche X = (x1, x2) tel que (a-lambda)x1 + b*x2 = 0
trouver_vecteur <- function(lambda, mat) {
  M <- mat - diag(lambda, nrow(mat))
  # On fixe arbitrairement x2 = 1 et on déduit x1 (si b != 0)
  # (a-lambda)x1 + b*1 = 0  => x1 = -b / (a-lambda)
  x1 <- -M[1,2] / M[1,1]
  v <- c(x1, 1)
  return(v / sqrt(sum(v^2))) # Normalisation
}

v1 <- trouver_vecteur(val_propres[1], A)
v2 <- trouver_vecteur(val_propres[2], A)

# 4. Reconstruction
P <- cbind(v1, v2)
D <- diag(val_propres)

print("Valeurs propres :")
```

```
## [1] "Valeurs propres :"
```

```r
print(val_propres)
```

```
## [1] 5 2
```

```r
print("Matrice de passage P :")
```

```
## [1] "Matrice de passage P :"
```

```r
print(P)
```

```
##             v1         v2
## [1,] 0.7071068 -0.4472136
## [2,] 0.7071068  0.8944272
```